



Reporte Técnico de Actividades Práctico-Experimentales Nro. 1 - 2

1. Datos de Identificación del Estudiante y la Práctica

Nombres de los estudiantes:	• Luis Anderson Carranza Quispe • Carlos Esteban Cevallos Soto • Jonathan Gerardo Contento Cuenca • Alvaro Yhalen Villafuerte Condoy
Asignatura	Matemáticas Discretas
Ciclo	1A
Unidad	Unidad 3: Grafos y Árboles
Resultado de aprendizaje de la unidad	Entiende los principios del análisis combinatorio que resuelven problemas reales de su cotidianidad, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001 fase 1 - 2
Título de la Práctica	Teoría de Grafos, Algoritmos de Búsqueda, Árboles y Aplicaciones Computacionales
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	29 de junio al 24 de julio de 2026
Horario	Lunes, Martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Laboratorio de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	12 horas (3 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Comprender los fundamentos de la teoría de grafos: definición, terminología (vértices, aristas, grado, adyacencia, incidencia), tipos de grafos (dirigidos, no dirigidos, ponderados, completos, bipartitos) y representaciones (matriz de adyacencia, lista de adyacencia).
- Aplicar algoritmos fundamentales de búsqueda en grafos: BFS (Búsqueda a lo Ancho), DFS (Búsqueda en Profundidad) y Dijkstra (camino más corto), comprendiendo su funcionamiento paso a paso.
- Entender la teoría de árboles: definición, propiedades, tipos de árboles (binarios, BST, AVL), recorridos (inorden, preorden, posorden) y aplicaciones en ciencias de la computación.
- Utilizar software de modelado de grafos (Graph Online, Gephi) para representar, visualizar y analizar redes y estructuras de datos.
- Integrar los conocimientos de las tres unidades mediante un proyecto que aplique la lógica matemática, el álgebra de Boole y la teoría de grafos a un problema computacional real.



unl

Universidad
Nacional
de Loja

3. Materiales, Reactivos

- Pizarra y marcadores de colores.
- Proyector multimedia.
- Hojas de trabajo con ejercicios de teoría de grafos y árboles.
- Guía paso a paso de los algoritmos BFS, DFS y Dijkstra.
- Plantillas para construcción de grafos y árboles.
- Investigación sobre aplicaciones de grafos en redes y optimización.

4. Equipos y herramientas

- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL
- Laboratorio de computación
- Software Graph Online (<https://graphonline.ru/es>) o Gephi para modelado de grafos
- Herramientas colaborativas: Google Drive, Wiki • Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

5. Procedimiento / Metodología

FASE 1: Fundamentos de la Teoría de Grafos (Semana 13: 29 junio - 03 julio 2026)

1. Se realiza una presentación participativa de los fundamentos de la teoría de grafos: definición formal de grafo, terminología clave (vértice/nodo, arista/lado, lazo, bucle, grado de un vértice, camino, ciclo, clique), clasificación de grafos (dirigidos/no dirigidos, ponderados/no ponderados, simples/multigrafos, completos, bipartitos, regulares, conexos).
2. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.
3. Práctica de representación de grafos: cada equipo representa el mismo grafo de tres formas diferentes: dibujo gráfico, matriz de adyacencia y lista de adyacencia. Comparan las ventajas y desventajas de cada representación en términos de eficiencia computacional y uso de memoria.
4. Los estudiantes investigan de forma autónoma sobre aplicaciones de grafos en redes de comunicaciones, optimización de rutas, bases de datos y redes sociales. Elaboran un mini informe individual con al menos 3 aplicaciones reales documentadas.
5. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).



FASE 2: Algoritmos de Búsqueda y Caminos Mínimos (Semana 14: 06-10 julio 2026)

1. El docente explica mediante Flipped Classroom los algoritmos fundamentales para grafos: BFS (Breadth-First Search / Búsqueda a lo Ancho), DFS (Depth-First Search / Búsqueda en Profundidad) y Dijkstra (Camino Más Corto), detallando su funcionamiento paso a paso, complejidad temporal y aplicaciones específicas.
2. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.
3. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.
4. Presentación grupal de resultados: cada equipo expone su análisis de optimización, explicando cómo los algoritmos de grafos pueden resolver problemas reales (rutas de entrega, diseño de redes, navegación GPS).
5. Discusión sobre la equidad digital y la importancia de la optimización de redes de comunicación para reducir la brecha digital en comunidades rurales del Ecuador. Reflexión sobre la responsabilidad ética en el diseño de algoritmos.

6. Resultados esperados

FASE 1

2. Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.

Los estudiantes, en equipos, construyen grafos a partir de especificaciones del lenguaje natural (ej: modelar la red de amigos de un grupo social, la red de transporte de una ciudad, la topología de una red de computadoras), identificando vértices, aristas, grados y propiedades estructurales.

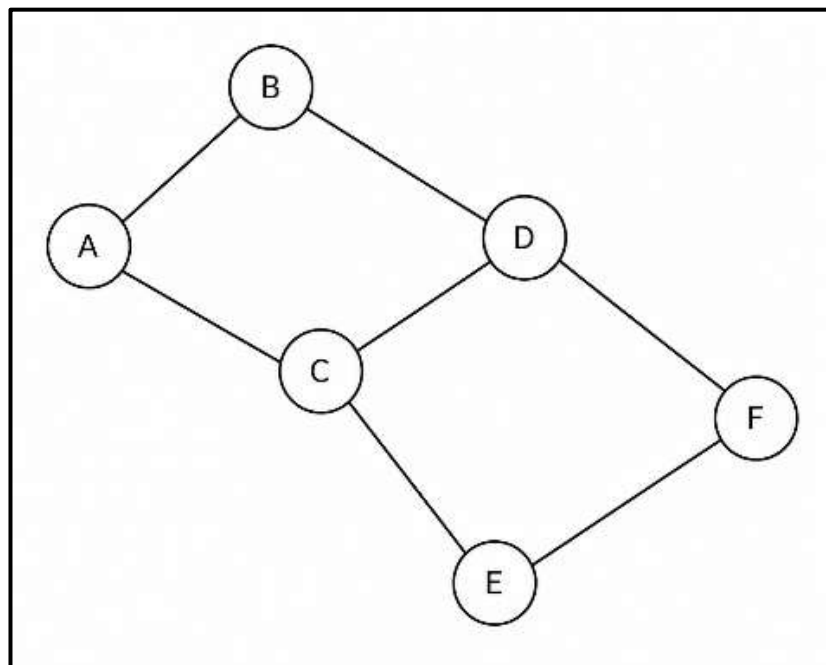
Se desea representar mediante un grafo una pequeña red de transporte urbano formada por seis estaciones de autobús. Cada estación estará representada por un vértice y cada carretera que conecta dos estaciones estará representada por una arista.

Las estaciones son:

- Terminal Central (A)
- Hospital (B)
- Universidad (C)
- Centro Comercial (D)
- Parque (E)
- Aeropuerto (F)

Las conexiones existentes son:

- Terminal Central – Hospital
- Terminal Central – Universidad
- Hospital – Centro Comercial
- Universidad – Centro Comercial
- Universidad – Parque
- Centro Comercial – Aeropuerto
- Parque – Aeropuerto



Identificación de los elementos

Vértices

Los vértices representan las estaciones de transporte.

$$V = \{A, B, C, D, E, F\}$$

Aristas

Las aristas representan las carreteras que conectan las estaciones.

$$E = \{(A, B), (A, C), (B, D), (C, D), (C, E), (D, F), (E, F)\}$$

Grado de cada vértice



Vértice	Conexiones	Grado
A	B, C	2
B	A, D	2
C	A, D, E	3
D	B, C, F	3
E	C, F	2
F	D, E	2

Comprobación:

$$2 + 2 + 3 + 3 + 2 + 2 = 14$$

Como: $2 \mid E \mid = 2(7) = 14$ la suma de los grados es correcta.

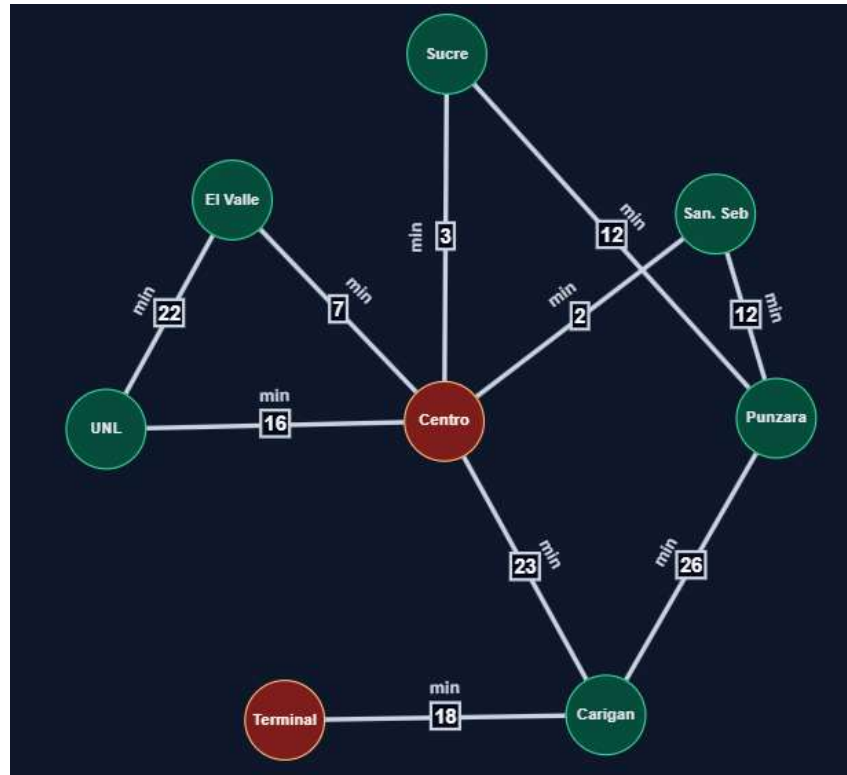
Propiedades estructurales

- Es un **grafo simple**, ya que no existen lazos ni aristas múltiples.
 - Es un **grafo no dirigido**, porque las carreteras pueden recorrerse en ambos sentidos.
 - Es un **grafo conexo**, ya que desde cualquier estación es posible llegar a cualquier otra siguiendo las carreteras.
 - Tiene **6 vértices**.
 - Tiene **7 aristas**.
 - No es un grafo completo, porque no todas las estaciones están conectadas directamente entre sí.
5. Modelado de una red sencilla mediante software Graph Online o Gephi: los equipos crean (Usar uno creado) un grafo representando una red de transporte en la ciudad de Loja, visualizándolo con el software y analizando sus propiedades (conectividad, caminos mínimos, nodos críticos).

Modelado de la red de transporte de la ciudad de Loja

1. Descripción del grafo

Se modeló la red como un grafo **no dirigido y ponderado**, compuesto por 8 vértices que representan 6 parroquias urbanas de Loja y 2 puntos clave de transporte, y 10 aristas que representan conexiones viales directas entre ellos.



Vértices: Centro (El Sagrario), Sucre, San Sebastián, El Valle, Punzara, Carigán, UNL, Terminal Terrestre

2. Metodología para obtener los pesos (tiempo de viaje)

Cada vértice se asoció a un punto de referencia real localizado en Google Maps:

Nodo	Punto de referencia	Coordenadas
Centro	Parque Central de Loja	-3.9966, -79.2016
Sucre	Iglesia El Pedestal	-3.9953, -79.2082
San Sebastián	Plaza San Sebastián	-4.0019, -79.2013
El Valle	Iglesia San Juan Bautista de El Valle	-3.9813, -79.2013
Punzara	Punzara Grande	-4.0194, -79.2231
Carigán	Parque/Laguna de Carigán	-3.9636, -79.2442
UNL	Universidad Nacional de Loja	-4.0329, -79.2025
Terminal Terrestre	Terminal "Reina del Cisne"	-3.9776, -79.2051

El tiempo de cada arista se calculó con la siguiente fórmula:

Paso 1 — Distancia en línea recta (aproximación plana, válida para distancias cortas dentro de una ciudad):

$$d_{recta} = R \times \sqrt{((\Delta lat)^2 + (\Delta lon \times \cos(lat_{prom}))^2)}, R = 6371 \text{ km}$$

Paso 2 — Ajuste a distancia vial real (las calles no son líneas rectas):

$$d_{real} = d_{recta} \times 1.3 (\text{factor de circuidad típico urbano})$$

Paso 3 — Tiempo estimado, con velocidad promedio urbana de 20 km/h (considera semáforos, tráfico y paradas):

$$t_{min} = \frac{d_{real}}{v} \times 60$$

Ejemplo de cálculo: Centro → Carigán

Datos:

- Centro: lat = -3.9966103°, lon = -79.2015815°
- Carigán: lat = -3.9635552°, lon = -79.2442450°

Paso 1 — Diferencias de coordenadas:

$$\Delta lat = -3.9635552 - (-3.9966103) = 0.0330551^\circ$$

$$\Delta lon = -79.2442450 - (-79.2015815) = -0.0426635^\circ$$

Paso 2 — Distancia en línea recta (latitud promedio = -3.9800828°, cos = 0.99758):

$$d_{recta} = 6371 \times \sqrt{\left(0.0330551 \times \frac{\pi}{180}\right)^2 + \left(-0.0426635 \times \frac{\pi}{180} \times 0.99758\right)^2} \\ \approx 5.992 \text{ km.}$$

Paso 3 — Distancia vial real:

$$d_{real} = 5.992 \times 1.3 = 7.790 \text{ km}$$

Paso 4 — Tiempo:

$$t = \frac{7.790}{20} \times 60 = 23.37 \text{ min} \approx \mathbf{23 \text{ min}}$$

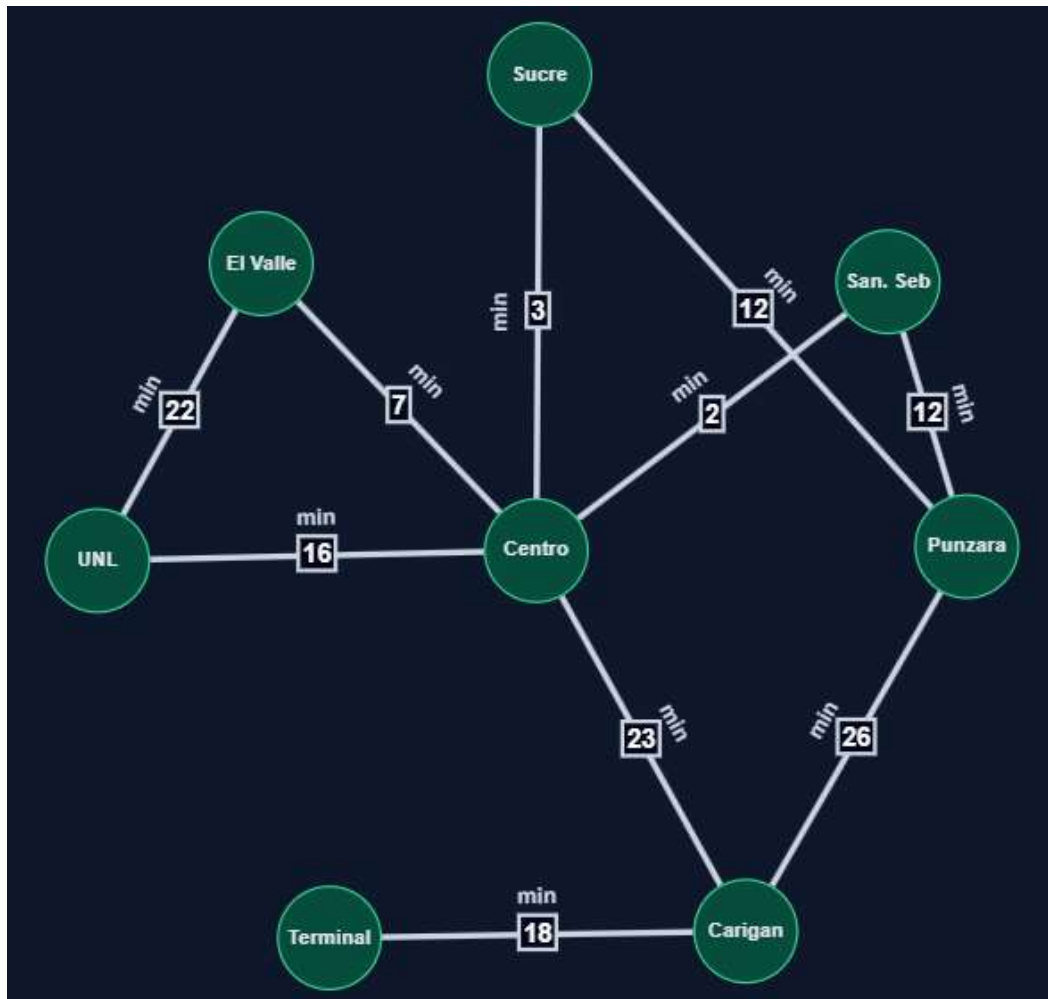


Este mismo procedimiento se replicó para las 10 aristas de la red.

3. Grafo resultante (vértices, aristas y pesos)

Arista	d recta (km)	d real (km)	Tiempo (min)
Centro–Sucre	0.76	0.98	3
Centro–San Sebastián	0.58	0.76	2
Centro–El Valle	1.69	2.20	7
Centro–Carigán	5.99	7.79	23
Centro–UNL	4.02	5.23	16
Sucre–Punzara	3.13	4.07	12
San Sebastián–Punzara	3.09	4.02	12
El Valle–UNL	5.71	7.42	22
Punzara–Carigán	6.60	8.59	26
Carigán–Terminal Terrestre	4.62	6.00	18

Grafo completo (vértices, aristas y pesos).

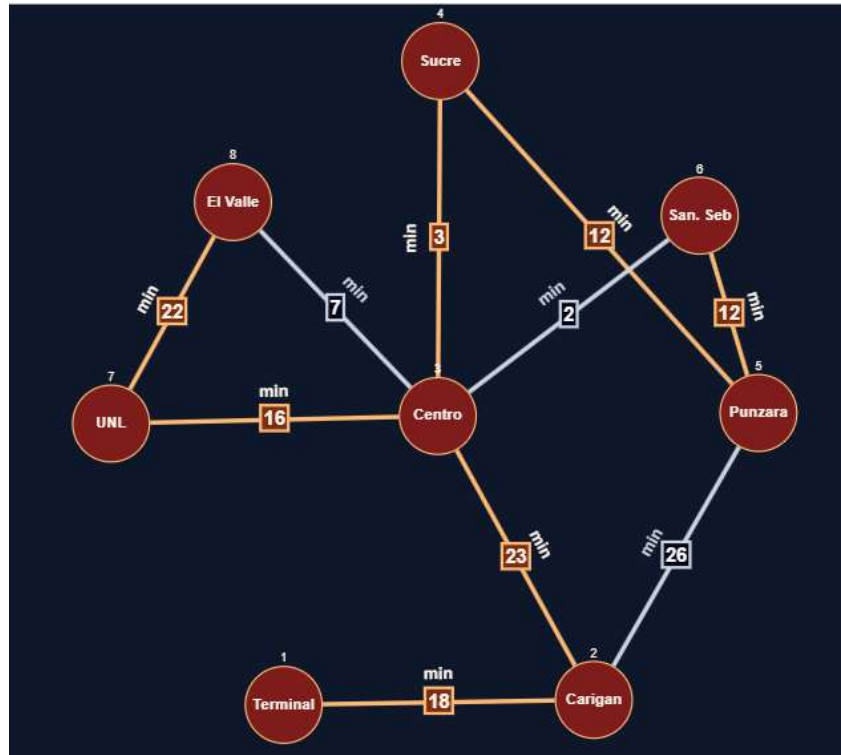


4. Análisis de propiedades

Conectividad

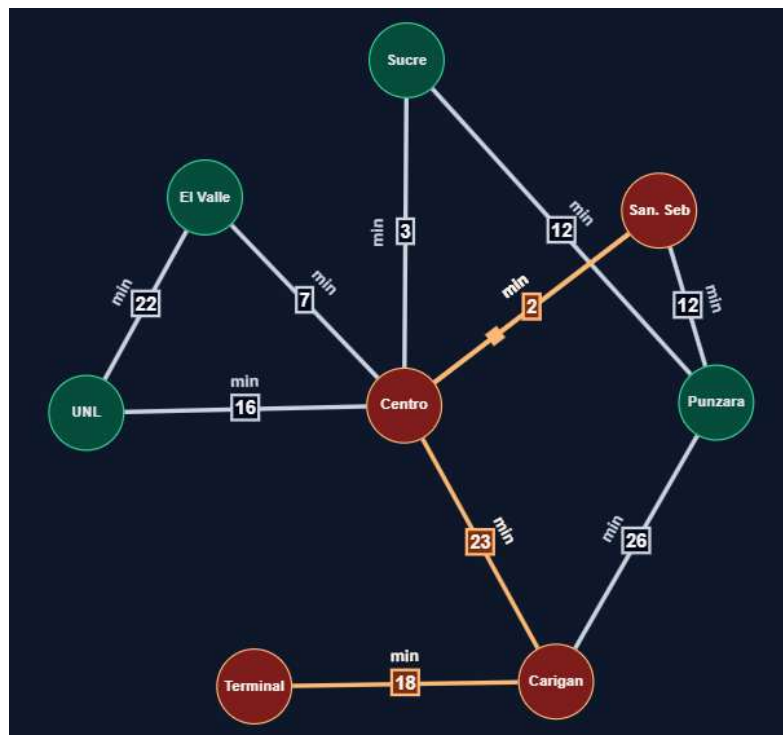
El grafo es **conexo**: no existen vértices aislados y desde cualquier nodo es posible llegar a cualquier otro siguiendo las aristas de la red. Se comprueba aplicando un recorrido BFS o DFS desde el vértice Centro: el algoritmo visita los 8 vértices sin dejar ninguno sin descubrir.

Búsqueda profunda de DFS:



Caminos mínimos (algoritmo de Dijkstra)

Se aplicó Dijkstra entre Terminal Terrestre y San Sebastián (llegada de un pasajero externo hacia un punto turístico/administrativo de la ciudad):



Terminal Terrestre → Carigán → Centro → San Sebastián

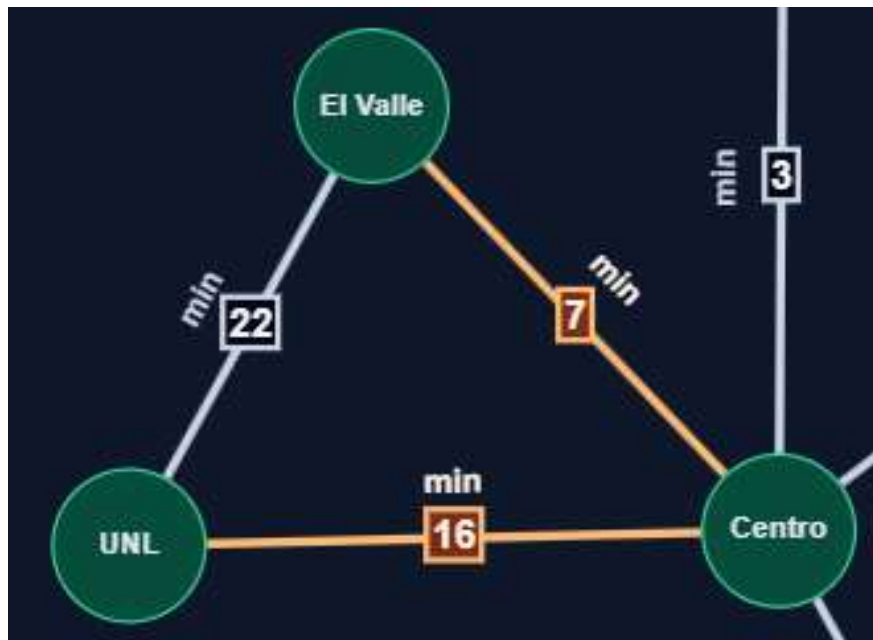
$18 + 23 + 2 = 43$ minutos

Se descartó la alternativa por Punzara (Terminal → Carigán → Punzara → San Sebastián = $18+26+12 = 56$ min) por tener mayor costo. El resultado confirma que, aunque Centro concentra las conexiones más cortas dentro del casco urbano, el tramo hacia Carigán (parroquia periférica) es el que más incrementa el tiempo total de viaje.

Nodos críticos (puntos de articulación)

Se identificaron dos nodos críticos — vértices cuya eliminación desconecta partes de la red:

- **Centro:** si se retira, El Valle y UNL quedan aislados del resto de la red, ya que su única vía de conexión pasa por este vértice.



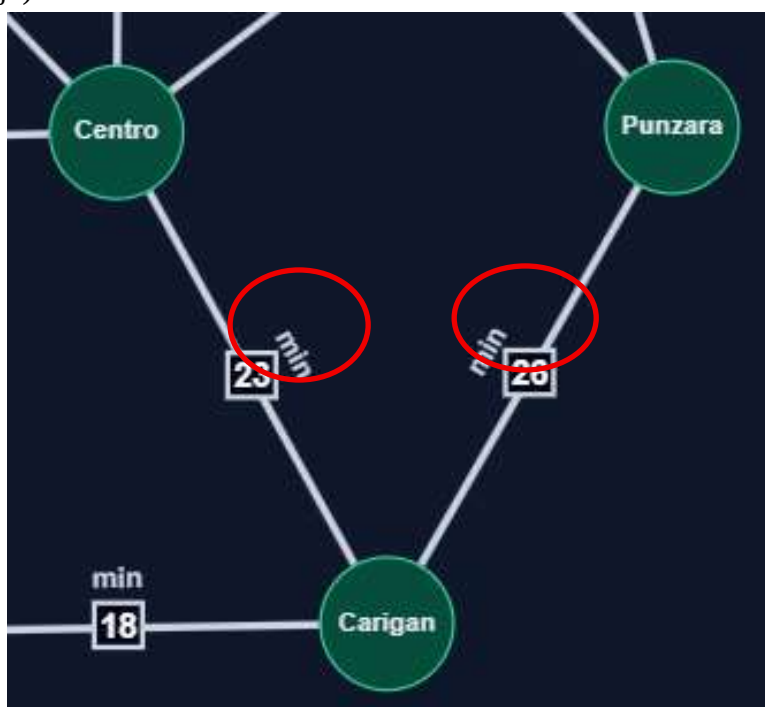
- **Carigán:** es el único punto de conexión hacia Terminal Terrestre; si se elimina, la terminal queda completamente incomunicada del resto de la ciudad.



Este hallazgo se mantiene independientemente de los pesos utilizados, porque depende únicamente de la topología del grafo (qué nodos están conectados entre sí), no de las distancias o tiempos asignados a cada arista.

5. Conclusión de optimización

Desde el punto de vista de planificación urbana, Centro y Carigán serían los puntos donde debería priorizarse el mantenimiento vial o la creación de rutas alternativas, ya que una falla en cualquiera de los dos comprometería la conectividad de sectores completos de la ciudad. Además, el análisis con datos reales evidencia que la conexión hacia Carigán es la más costosa en tiempo de toda la red (23 y 26 minutos en sus dos aristas), lo que la convierte en un cuello de botella tanto estructural (nodo crítico) como operativo (mayor tiempo de viaje).



FASE 2

2. Taller colaborativo de aplicación de algoritmos: los equipos ejecutan manualmente los algoritmos BFS, DFS y Dijkstra sobre grafos de ejemplo (no dirigidos para BFS/DFS, ponderados para Dijkstra), documentando cada paso: cola/pila de nodos visitados, orden de descubrimiento, distancias acumuladas y caminos resultantes.

EJERCICIO 1: Algoritmo BFS (Búsqueda a lo Ancho)

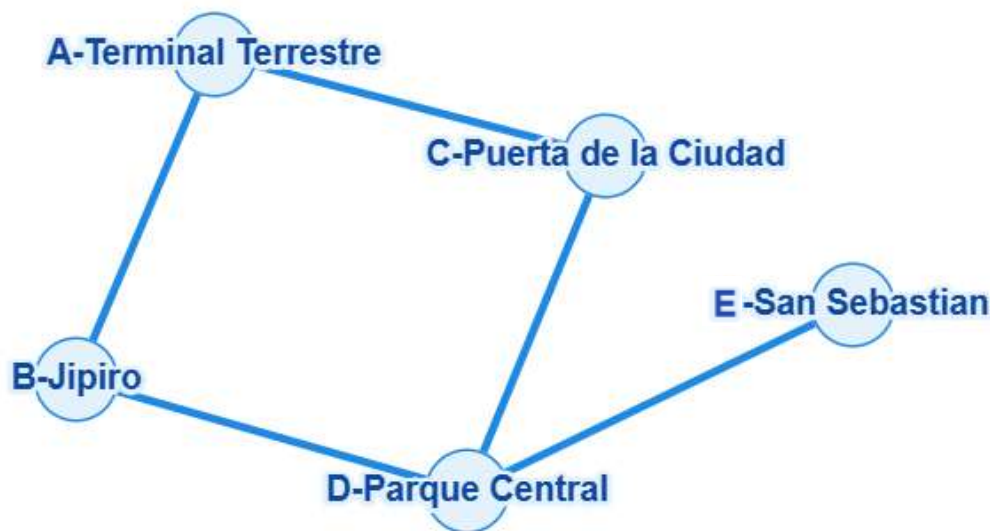
Escenario: Red de transporte en los puntos principales de la ciudad (Grafo No Dirigido).

Objetivo del algoritmo.

El algoritmo BFS (*Breadth-First Search*) tiene como objetivo explorar un grafo visitando primero todos los nodos que se encuentran a una misma distancia (o nivel) del punto de origen, antes de proceder a avanzar de forma más profunda. En otras palabras podemos decir que funciona como una "ola" expansiva. Con lo cual para lograr este

comportamiento ordenado por capas, utiliza una estructura de datos lineal llamada **Cola**, la cual se rige bajo el principio FIFO (*First In, First Out*: el primero en entrar es el primero en salir). Es ideal para encontrar el número mínimo de conexiones entre dos puntos en una red no ponderada.

Representación Visual del Grafo:



Descripción paso por paso:

- **Estado Inicial (Origen):** Nos posicionamos en el nodo inicial, el **Terminal Terrestre (A)**. Lo marcamos como descubierto.
 - **Estado de la Cola:** [A]
 - **Nodos Visitados:** A
 - **Orden de Descubrimiento Provisional:** A
- **Paso 1:** Extraemos el primer nodo al frente de la cola, que es **A**. Analizamos de manera natural sus conexiones directas no visitadas: el **Parque Jipiro (B)** y la **Puerta de la Ciudad (C)**. Colocamos ambos nodos en la cola para que esperen su turno de inspección.
 - **Estado de la Cola:** [B, C]
 - **Nodos Visitados:** A, B, C
 - **Orden de Descubrimiento:** A, B, C
- **Paso 2:** Extraemos el nodo que ahora está al frente de la cola: **B**. Revisamos sus vecinos directos. Su única conexión hacia un nodo no visitado es el **Parque Central (D)**. Colocamos a **D** al final de la cola.
 - **Estado de la Cola:** [C, D]



unl

Universidad
Nacional
de Loja

- **Nodos Visitados:** A, B, C, D
- **Orden de Descubrimiento:** A, B, C, D
- **Paso 3:** Extraemos el siguiente nodo en la cola, la **Puerta de la Ciudad (C)**. Revisamos sus vecinos directos. Está conectado con **D**, pero como **D** ya ha sido descubierto previamente (está en la cola), el algoritmo lo ignora para evitar bucles redundantes.
 - **Estado de la Cola:** [D]
 - **Nodos Visitados:** A, B, C, D
 - **Orden de Descubrimiento:** A, B, C, D
- **Paso 4:** Extraemos el nodo **Parque Central (D)**. Evaluamos sus enlaces y encontramos el último punto sin visitar de la ciudad: **San Sebastián (E)**. Lo mandamos al final de la cola.
 - **Estado de la Cola:** [E]
 - **Nodos Visitados:** A, B, C, D, E
 - **Orden de Descubrimiento:** A, B, C, D, E
- **Paso 5:** Extraemos el nodo **E**. Al examinar sus conexiones, verificamos que no existen más nodos adyacentes por descubrir. La cola queda completamente vacía.
 - **Estado de la Cola:** []
 - **Caminos Resultantes:** Se ha explorado con éxito la red completa.
 - **Orden de Descubrimiento Final:** $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

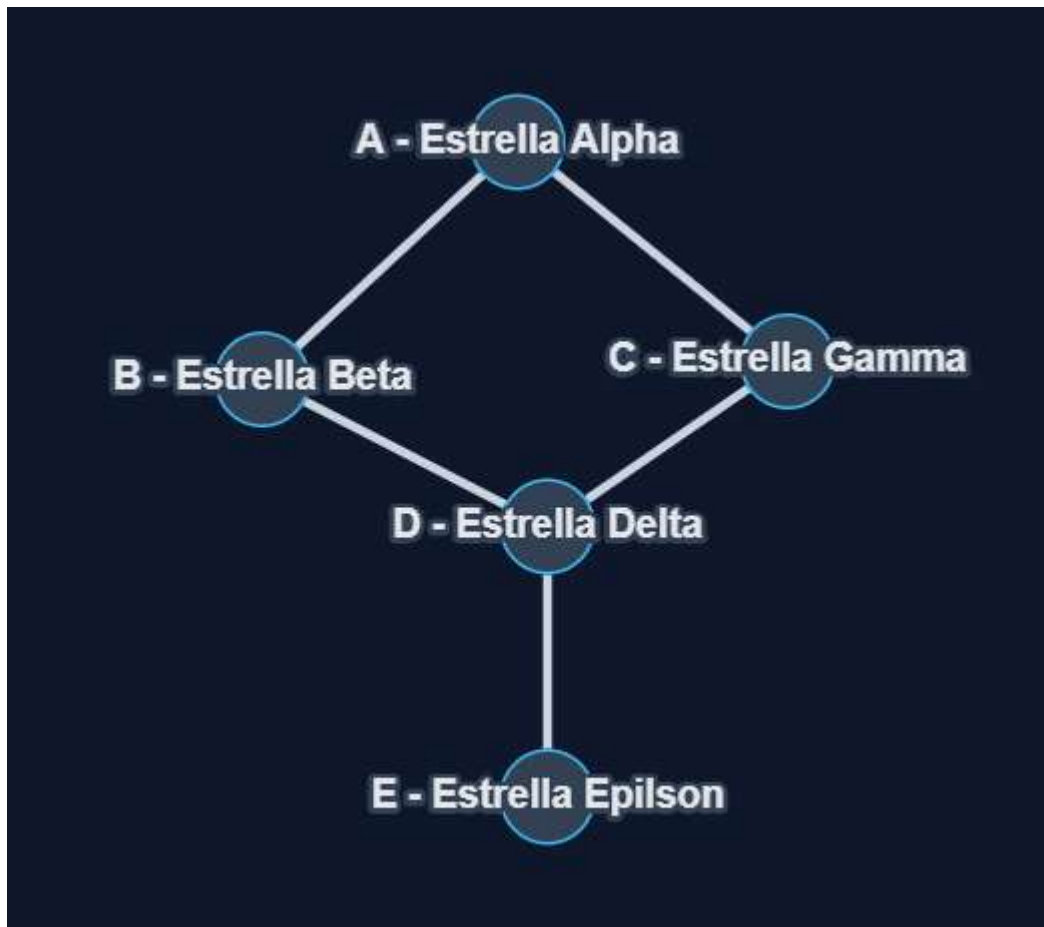
EJERCICIO 2: Algoritmo DFS (Búsqueda en Profundidad)

Escenario: Mapa de constelaciones y alineación de estrellas (Grafo No Dirigido).

Objetivo del algoritmo:

A diferencia del enfoque por capas, el algoritmo DFS (*Depth-First Search*) busca explorar un camino único de forma tan profunda como sea posible antes de verse obligado a retroceder. Funciona de manera similar a adentrarse en un laberinto sin mirar a los lados. Para recordar las bifurcaciones y saber a dónde regresar, utiliza internamente una estructura de **Pila**, la cual opera bajo el principio LIFO (*Last In, First Out*: el último en entrar es el primero en salir). Cuando el algoritmo llega a un "callejón sin salida" (un nodo sin vecinos nuevos), realiza un proceso de retroceso (*backtracking*) sacando elementos de la pila para retomar las rutas que dejó pendientes.

Representación Visual del Grafo:



Descripción paso por paso:

- **Estado Inicial (Origen):** Iniciamos la observación astronómica en la **Estrella Alpha (A)**. Colocamos a **A** en la pila y lo marcamos como visitado.
 - **Estado de la Pila:** [A]
 - **Orden de Descubrimiento Provisional:** A
- **Paso 1:** Sacamos a **A** de la pila para inspeccionar sus adyacencias libres: la Estrella Beta (B) y la Estrella Gamma (C). Guardamos a **C** en la base de la pila para el futuro e introducimos a **B** arriba, decidiendo avanzar primero por su línea de luz.
 - **Estado de la Pila:** [C, B]
 - **Orden de Descubrimiento:** A, B
- **Paso 2:** Extraemos el nodo superior de la pila, que es **B**. Siguiendo la trayectoria recta y profunda, cruzamos desde **B** directamente hacia su vecino libre, la **Estrella Delta (D)**. Añadimos a **D** en la parte superior de la pila.
 - **Estado de la Pila:** [C, D]



- **Orden de Descubrimiento:** A, B, D
- **Paso 3:** Sacamos a **D** de la pila. Desde este punto, el algoritmo observa dos estrellas conectadas: Gamma (C) y Epsilon (E). Siguiendo con la regla de ir lo más profundo posible, el algoritmo prioriza avanzar hacia la **Estrella Epsilon (E)**, metiéndola en la pila.
 - **Estado de la Pila:** [C, E]
 - **Orden de Descubrimiento:** A, B, D, E
- **Paso 4 (Proceso de Retroceso / Backtracking):** Sacamos a **E** de la pila. Al analizar el entorno de la Estrella Epsilon (E), descubrimos que no tiene más conexiones hacia estrellas nuevas; es un extremo final. En lenguaje natural, el algoritmo "da la vuelta" y retrocede en su historial para ver qué dejó pendiente. Al revisar la pila, el siguiente elemento disponible es la **Estrella Gamma (C)**.
 - **Estado de la Pila:** [C]
 - **Orden de Descubrimiento:** A, B, D, E
- **Paso 5:** El algoritmo viaja físicamente a evaluar el nodo restante en la pila, la **Estrella Gamma (C)**. Al posicionarse en ella, constata que todos sus vecinos estructurales (A y D) ya fueron visitados en los pasos anteriores. Al no haber nada más que descubrir, limpia la pila.
 - **Estado de la Pila:** []
 - **Caminos Resultantes:** Se completó el trazado profundo de la constelación.
 - **Orden de Descubrimiento Final:** $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

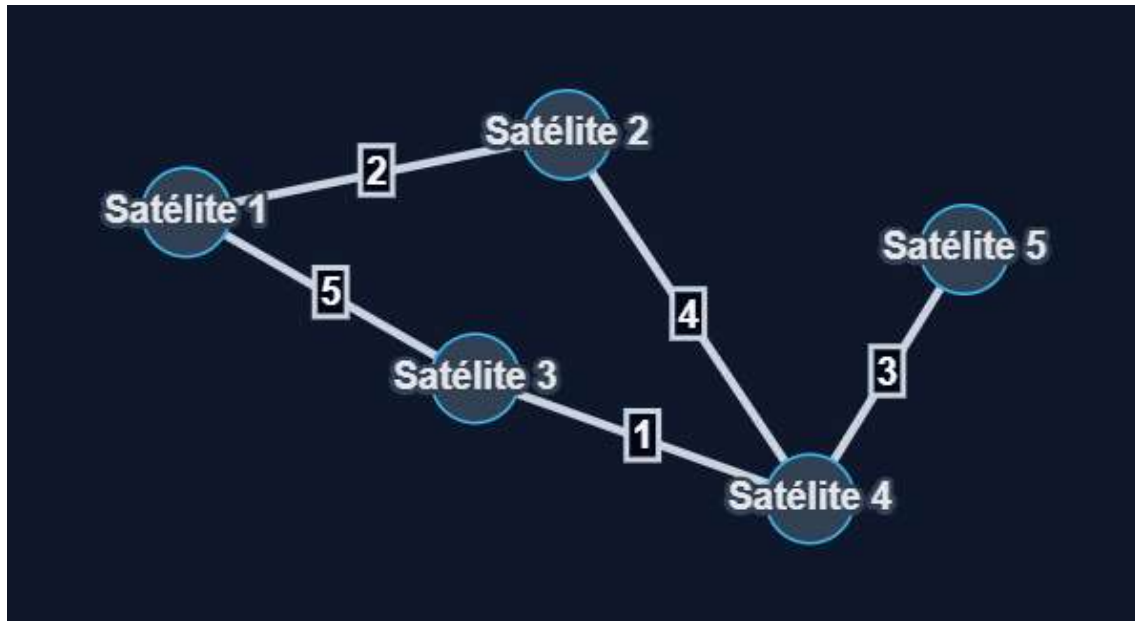
EJERCICIO 3: Algoritmo de Dijkstra (Camino Más Corto)

Escenario: Enrutamiento de datos en una Red de Satélites de Telecomunicaciones (Grafo Ponderado).

Objetivo del algoritmo:

El algoritmo de Dijkstra se encarga de hallar la ruta óptima absoluta (el camino con el menor costo o latencia total acumulada) entre un nodo inicial y todos los demás destinos dentro de un grafo ponderado (donde los enlaces poseen valores numéricos o pesos). Funcionando bajo una estrategia llamada "voraz" (*greedy*), manteniendo de forma constante una tabla de registros con las distancias mínimas conocidas. El algoritmo evalúa paso a paso el nodo libre que tenga asignado el menor costo acumulado, revisa si a través de él puede mejorar (reducir) los tiempos de viaje de sus vecinos, actualiza los valores de la tabla si el nuevo camino es más eficiente (proceso llamado *relajación*) y avanza de forma matemática segura.

Representación Visual del Grafo:



Descripción paso por paso:

- **Fase de Control Inicial (Tabla de costos acumulados desde S1):**
 - Distancia al nodo origen $S1 = 0$
 - Distancia al resto de satélites provisionalmente es infinita, puesto que no conocemos la distancia exacta. $(\infty) = S2 = \infty, S3 = \infty, S4 = \infty, S5 = \infty$.
 - Conjunto de satélites no visitados de forma definitiva: $\{S1, S2, S3, S4, S5\}$.
- **Iteración 1 (Nodo Actual Evaluado: S1 | Costo Acumulado = 0):**
 - Inspeccionamos los satélites al alcance directo del emisor S1:
 - Hacia **S2**: El costo es de 0 (origen) + 2 (enlace) = 2. Como el número 2 es menor que ∞ , actualizamos el registro de S2 a 2.
 - Hacia **S3**: El costo es de 0 (origen) + 5 (enlace) = 5. Como $5 < \infty$, actualizamos el registro de S3 a 5.
 - **Tabla de Distancias Actualizada:**

Satélite	Distancia
S1	0 (Origen)
S2	2
S3	5
S4	∞
S5	∞

- **Iteración 2 (Nodo Actual Evaluado: S2 | Costo Acumulado = 2):**

- De los satélites libres, el que tiene menor valor en la tabla es **S2** (costo de 2). Nos paramos en S2. Evaluamos su enlace de salida:
 - Hacia **S4**: El costo acumulado es la distancia de S2 más el peso del canal hacia S4: $2 + 4 = 6$. Como $6 < \infty$, registramos provisionalmente que llegar a S4 cuesta **6**.
- **Tabla de Distancias Actualizada:**

Satélite	Distancia
S1	0 (Origen)
S2	2
S3	5
S4	6
S5	∞

- **Iteración 3 (Nodo Actual Evaluado: S3 | Costo Acumulado = 5):**

- Entre los satélites libres restantes (S3 con costo 5 y S4 con costo 6), elegimos el menor: **S3**. Nos paramos en él y analizamos su canal:
 - Hacia **S4**: El costo acumulado calculado a través de esta vía alternativa es la distancia de S3 más su enlace a S4: $5 + 1 = 6$.
 - **Análisis:** El nuevo costo (6) empata de forma exacta con la ruta que se tenía guardada desde S2. Como no es estrictamente menor, por estabilidad el algoritmo mantiene el registro previo intacto.

- **Tabla de Distancias Actualizada:**

Satélite	Distancia
S1	0 (Origen)
S2	2
S3	5
S4	6
S5	∞

- **Iteración 4 (Nodo Actual Evaluado: S4 | Costo Acumulado = 6):**

- Tomamos el satélite **S4** que posee un valor acumulado de 6. Evaluamos su paso hacia el destino final:
 - Hacia **S5**: El costo acumulado es la distancia fija de S4 más el peso del último tramo: $6 + 3 = 9$. Al ser menor que infinito, lo guardamos fijamente en la tabla.
- **Tabla de Distancias Actualizada:**



Satélite	Distancia
S1	0 (Origen)
S2	2
S3	5
S4	6
S5	9

- **Iteración 5 (Nodo Actual Evaluado: S5 | Costo Acumulado = 9):**
 - Se selecciona el último nodo libre de la lista, el satélite de destino **S5**, fijando su valor definitivo.
 - **Distancias Acumuladas Finales:**

$$S1 = 0; S2 = 2; S3 = 5; S4 = 6; S5 = 9$$

- **Camino Resultante Óptimo:** La ruta más eficiente para la transmisión de datos es viajar desde el origen $S1 \rightarrow S2 \rightarrow S4 \rightarrow S5$, logrando una latencia mínima optimizada de **9 milisegundos**.

3. Los equipos modelan redes mediante software (Graph Online, Gephi u otros), ingresando grafos ponderados y ejecutando el algoritmo de Dijkstra para encontrar el camino más corto entre pares de nodos. Comparan los resultados manuales con los obtenidos por el software.

Modelado de redes con grafos: routers de una red universitaria

El objetivo del ejercicio es modelar mediante un grafo ponderado una red de datos de la universidad Nacional de Loja. Cada vértice representará un router ubicado en un punto diferente de todo el campus de la Universidad, y cada arista será el enlace que existe entre dos routers, ponderado con latencia. Sobre este grafo se aplica el algoritmo Dijkstra para encontrar el camino de menor latencia entre dos routers y luego comparar el resultado obtenido de manera manual con el que entrega un software de grafos (Graph Online)

1. Modelado del grafo

El grafo cuenta con 10 vértices (routers) distribuidos en distintos edificios del campus:

Nodo	Ubicación en el campus
R1	Biblioteca
R2	Rectorado
R3	Facultad de la Energía, las Industrias y los Recursos Naturales no Renovables
R4	Facultad Agropecuaria y de Recursos Naturales Renovables
R5	Facultad de la Educación, el Arte y la Comunicación



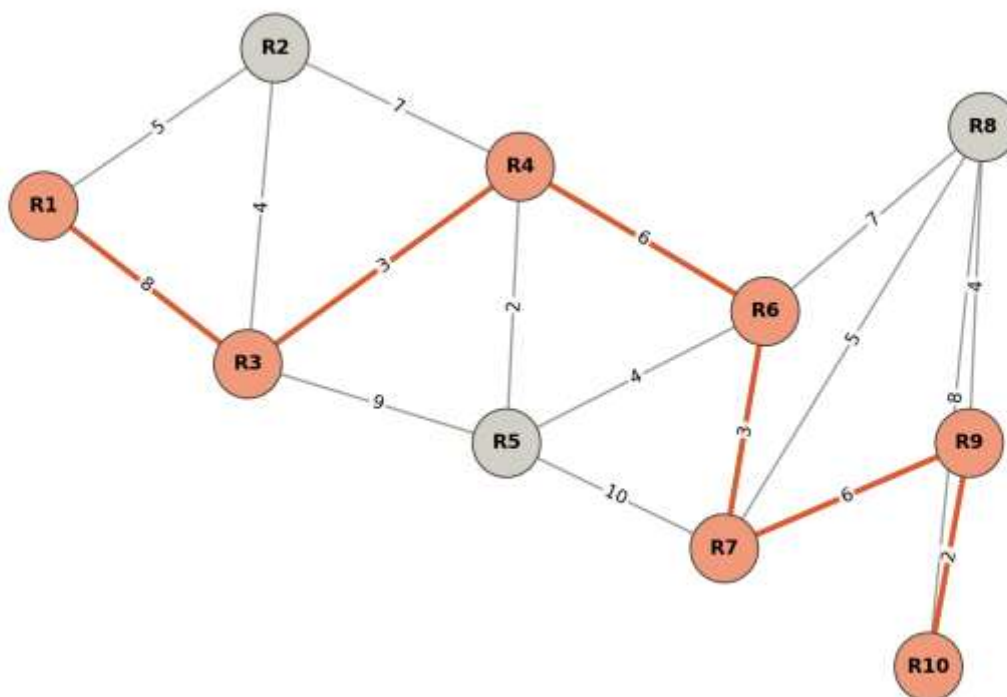
R6	Facultad Jurídica, Social y Administrativa
R7	Facultad de la Salud Humana
R8	Centro deportivo
R9	Unidad de Educación a Distancia en Línea
R10	Bienestar Universitario

Las aristas representan los enlaces entre routers, con pesos aleatorios que simulan la latencia en milisegundos (ms):

Arista (enlace)	Latencia (ms)
R1–R2	5
R1–R3	8
R2–R3	4
R2–R4	7
R3–R4	3
R3–R5	9
R4–R5	2
R4–R6	6
R5–R6	4
R5–R7	10
R6–R7	3
R6–R8	7
R7–R8	5
R7–R9	6
R8–R9	4
R8–R10	8
R9–R10	2

Representación gráfica del grafo

Red de routers universitarios (latencia en ms) — camino más corto R1→R10 resaltado



En rojo se resalta el camino más corto encontrado entre R1 y R10, desarrollado en la siguiente sección.

2. Ejercicio: camino más corto de R1 (Biblioteca) a R10 (Bienestar Universitario).

Aplicación manual del algoritmo de Dijkstra

Se inicializa la distancia del nodo origen R1 en 0 y la de todos los demás nodos en infinito. En cada paso se visita el nodo no visitado con menor distancia acumulada y se actualizan las distancias de sus vecinos:

Paso	Nodo visitado (distancia)	Distancias actualizadas
1	R1 (0)	R2 = 5, R3 = 8
2	R2 (5)	R4 = 12
3	R3 (8)	R4 = 11 (mejora), R5 = 17
4	R4 (11)	R5 = 13 (mejora), R6 = 17
5	R5 (13)	R7 = 23
6	R6 (17)	R7 = 20 (mejora), R8 = 24
7	R7 (20)	R9 = 26
8	R8 (24)	R10 = 32
9	R9 (26)	R10 = 28 (mejora)
10	R10 (28)	— fin del algoritmo



Resultado manual

Camino más corto: $R1 \rightarrow R3 \rightarrow R4 \rightarrow R6 \rightarrow R7 \rightarrow R9 \rightarrow R10$

Latencia total: $8 + 3 + 6 + 3 + 6 + 2 = 28 \text{ ms}$

3. Comparación con software (Graph Online / Gephi)

Para validar el resultado manual con una herramienta de software, se siguieron estos pasos:

- Se ingresó el mismo grafo ponderado en Graph Online, creando los 10 nodos R1–R10 y uniendo los vértices con las 17 aristas con los pesos indicados.
- Se ejecutó la función del algoritmo de Dijkstra integrada en la herramienta, seleccionando R1 como origen y R10 como destino.
- El software devolvió el mismo camino (R1-R3-R4-R6-R7-R9-R10) y el mismo costo total (28 ms), confirmando que el cálculo manual es correcto.

Tabla comparativa

	Cálculo manual	Software (Graph Online / Gephi)
Camino más corto	R1-R3-R4-R6-R7-R9-R10	R1-R3-R4-R6-R7-R9-R10
Latencia total	28 ms	28 ms



unl

Universidad
Nacional
de Loja

Bibliografía

Básica:

Oscar Levin, University of Northern Colorado (2022). Discrete Mathematics: An Open Introduction – 4th Edition. Universidad Tecnológica de Pereira. ISBN: 9781534970748.

Harris Kwon (2024). A Spiral Workbook for Discrete Mathematics. Milne Open Textbooks. ISBN: 978-1-956862-01-0.

Complementaria:

André Greiner-Petter (2023). Making Presentation Math Computable. Springer. ISBN: 978-3 658-40472-7.

Kevin Powell (2025). Discrete Perspectives in Mathematics – Second Edition. APE Nro. 003 — Unidad 3: Grafos y Árboles

Kazushi Ikeda et al. (2024). Advanced Mathematical Science for Mobility Society. Springer. ISBN: 978-981-99-9771-8. Recursos de Internet: • PLATZI. (2025). Curso de Matemáticas Discretas.

Juan Miguel León Rojas. Cuestiones Matemáticas Discretas. • Oscar Levin. Discrete Mathematics (Levin). <https://discrete.openmathbooks.org>